

Topic 3.7

3.7: Default Formal Parameters

C++ allows some actual parameters to be omitted when invoking a function.

In these situations the formal parameters corresponding to the omitted actual parameters receive a default value specified in the function definition. These formal parameters are known as *defaultable parameters*.

Defaultable parameters must appear at the end of the parameter list. There can be no non-defaultable parameters after the first defaultable parameter.

Here's a simple example:

```
// Print a sentence. The defaults are to print
// the sentence in the 12-point Times New Roman
// font, with no italics, no bold, and no
// underline.

int print_sentence(char *sentence,
    FONT font = TIMES_NEW_ROMAN,
    unsigned font_size = 12,
    unsigned italic = 0,
    unsigned bold = 0,
    unsigned underline = 0
);

// Print a sentence using all defaults.

print_sentence( "Hello world!");

// Print a sentence using SYMBOL 18-point font.

print_sentence("This uses SYMBOL font.",
    SYMBOL, 18);

// Print a sentence using ARIAL 14-point font
// in italics and underlined.

print_sentence("Hello everyone.", ARIAL, 14, 1, 0, 1);
```

The next example program, `distance1.cpp`, illustrates *operator overloading*. It defines an additional operator that works on English distances, using ordinary [structs](#). The addition operator takes two operands, one on the left of the '+' and one on the right. When defining an operator, we really write a function that has a special name:

`Distance operator+ ( Distance lhs, Distance rhs );`

Here, the function name is `operator+`. The two arguments are *lhs* and *rhs*, which refer to the left-hand-side and right-hand-side (the operands on either side of the operator), respectively. Having written this function, you can actually invoke it using functional notation:

```
Distance a, b, sum;
sum = operator+( a, b );
```

However, this defeats the purpose because the functional notation tends to clutter the code and is much less readable than conventional infix mathematical notation. Instead, write

```
sum = a + b;
```

and the compiler locates our function and invokes it with *a* as its first actual parameter and *b* as its second actual parameter.

When writing binary operator functions, the first formal parameter always represents the left hand operand, and the second formal parameter always represents the right hand operand.

```
// FILENAME:   distance1.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       4/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    distance1.cpp
// PURPOSE:
//   Provide a type for English distances.
//   Illustrate operator overloading.

#include <iostream>
#include <stdlib.h>

using namespace std;

typedef struct { int feet;
                double inches;
            } Distance;

Distance operator + ( Distance &lhs,
                    Distance &rhs );

int main(void)
{
    Distance a = {16,5.0};
    Distance b = {17,9.0};
    Distance total;

    total = a + b;
    cout << "The total distance is " << total.feet
         << " " << total.inches << "\n\n";

    return 0;
}

Distance operator + ( Distance &lhs,
                    Distance &rhs )
{
    Distance sum;
```

```

sum.feet = lhs.feet + rhs.feet;
sum.inches = lhs.inches + rhs.inches;
if (sum.inches >= 12.0)
{
    sum.inches -= 12.0;
    ++sum.feet;
}
return sum;
}

```

Output from distanc1.cpp:

```
The total distance is 34'2"
```

The following example shows how to turn our [Distance](#) type into a *class*. A primary difference is that the `operator+` function is now a member function. When a binary operator is written as a member function the invoking object always represents the left hand operand, and the first (only) formal parameter always represents the right hand operand.

In this example we also see how to overload the stream insertion and stream extraction operators. There is a canonical pattern we must follow here, as illustrated by the declaration

```
ostream& operator << ( ostream& str, Distance& d );
```

The existence of this function allows us to output a [Distance](#):

```

Distance my_height(5,8); // As Garfield says,
                        // I'm undertall.
cout << my_height;

```

The first formal parameter to the stream insertion operator function is a reference to the stream to which we want the output to go. Normally we will output to `cout`, but we can use our function to output to any stream, including to files we open as streams. This will be true for all overloaded stream insertion operators.

The second formal parameter is the object being output, usually passed by reference for efficiency. The return type of the function is a reference to an *ostream* (`ostream ref`). This enables our function to be invoked in a cascade just as we can with pre-existing stream insertion operators:

```
cout << my_height << " and I'm too short. ";
```

I couldn't apply the second `<<` above if my overloaded insertion operator didn't return a reference to the same stream we just output the object to. The same concept applies to the *stream extraction* operator.

Note that when we write a function that returns a reference, it looks like the function body is returning an object, not a reference. That's because programmers cannot create references, only the compiler

can. We simply indicate to the compiler that we would like it to generate a reference and it does this for us:

```
any_type& abc( 0 )
{
    return  any_type_object;
}
```

While it looks like we are returning object *any_type_object* of type *any_type*, in reality the compiler returns a *reference* to that object because of our declaration that this function return a reference.

Note that it is impossible for the stream insertion operator to be a member of the *Distance* class. We want it to be an operator, but remember that the left hand operand of the operator would then have to be an object of the class, for example, a *Distance*. However, for the stream insertion operator, the left hand operand is always an *ostream*. Thus, the only class this function could be a member of is *ostream*. However, we write so many versions of the stream insertion and stream extraction operators that it is impractical to modify those classes each time. Therefore, we must make these operators *friend functions*.

```
// FILENAME:  distance2.cpp
// PROGRAMMER: Gary J. Blair
// DATE:      4/11/1996
// COMPILER:  MS Visual C++ 2008 Express Edition
// REQUIRED:   distance2.cpp
// PURPOSE:
//   Provide a type for English distances.
//   Illustrate creation of new stream insertion
//   and stream extraction operators.

#include <iostream>
#include <stdlib.h>

using namespace std;

typedef struct {
    int feet;
    double inches;
} Distance;

Distance operator + ( Distance &lhs,
                    Distance &rhs );
ostream& operator << ( ostream& str, Distance& d );
istream& operator >> ( istream& str, Distance& d );

int main(void)
{
    Distance  a = {16,5.0};
    Distance  b = {17,9.0};
    Distance  total;

    total = a + b;
    cout<< a << " + " << b << " = " << total
         << endl;

    cout << "Enter 2 distances ";
    cin >> a >> b;
    total = a + b;
    cout << a << " + " << b << " = " << total
```

```

        << endl;

    return 0;
}

```

Output from distanc2.cpp:

```

16'5" + 17'9" = 34'2"
Enter 2 distances 6'9.25" 3'10.5"
6'9.25" + 3'10.5" = 10'7.75"

```

```

Distance operator + ( Distance &lhs,
                    Distance &rhs )
{
    Distancesum;

    sum.feet = lhs.feet + rhs.feet;
    sum.inches = lhs.inches + rhs.inches;
    if (sum.inches >= 12.0)
    {
        sum.inches -= 12.0;
        ++sum.feet;
    }
    return sum;
}

ostream& operator << ( ostream& str, Distance& d )
{
    str << d.feet << "'" << d.inches << "\"";
    return str;
}

istream& operator >> ( istream& str, Distance& d )
{
    str >> d.feet;
    if (str.get() != '\n')
    {
        cerr << "*** Error extracting Distance.\n";
        exit(1);
    }
    str >> d.inches;
    if (str.get() != '\n')
    {
        cerr << "*** Error extracting Distance.\n";
        exit(1);
    }
    return str;
}

```

```

// FILENAME: distance3.cpp
// PROGRAMMER: Gary J. Blair
// DATE: 4/11/1996
// COMPILER: MS Visual C++ 2008 Express Edition
// REQUIRED: distance3.cpp, distance.h, distance.cpp
// PURPOSE:
// Provide a class for English distances.

```

```

// Illustrate operator overloading in classes.
// Illustrate friend functions.

#include <iostream>
#include <stdlib.h>
#include "distance.h"

using namespace std;

int main(void)
{
    Distance  a;
    Distance  b(3,6.5);
    Distance  total;

    total = a + b;
    cout << a << " + " << b << " = " << total
         << endl;

    cout << "Enter 2 distances ";
    cin >> a >> b;
    total = a + b;
    cout << a << " + " << b << " = " << total
         << endl;

    return 0;
}

0'0" + 3'6.5" = 3'6.5"
Enter 2 distances 5'5" 9'9"
5'5" + 9'9" = 15'2"

// FILENAME:  distance.h
// PROGRAMMER: Gary J. Blair
// DATE:      4/11/1996
// COMPILER:  MS Visual C++ 2008 Express Edition
// REQUIRED:   distance3.cpp, distance.h, distance.cpp
// PURPOSE:
// Header file for Distance class.

using namespace std;

class Distance
{
public:

    // The no argument constructor initializes
    // a Distance to 0'0".

    Distance();

    // Initialize a Distance to the specified value.

    Distance( int ft, double in );

    // Define + that operates on
    // Distance + Distance.

    Distance operator + ( Distance &rhs );

    // Define + that operates on Distance + int,
    // which means add feet.

    Distance operator + ( int rhs );

    // Define + that operates on Distance + double,
    // which means add inches.

```

```

    Distance operator + ( double rhs );

// Define + that operates on int + Distance,
// which means add feet.

    friend Distance operator + ( int lhs,
                                Distance &rhs );
// Define + that operates on double + Distance,
// which means add inches.

    friend Distance operator + ( double lhs,
                                Distance &rhs );

// Define stream insertion operator
// for Distances.

    friend ostream& operator << ( ostream& str,
                                   Distance& d );

// Define stream extraction operator
// for Distances.

    friend istream& operator >> ( istream& str,
                                   Distance& d );

private:
    int    feet;
    double inches;
};

// FILENAME:  distance.cpp
// PROGRAMMER: Gary J. Blair
// DATE:      96/04/11
// COMPILER:  MS Visual C++ 2008 Express Edition
// REQUIRED:   distance3.cpp, distance.h, distance.cpp
// PURPOSE:
// Implement the Distance class.

#include <iostream>
#include <stdlib.h>
#include "distance.h"

// The no argument constructor initializes
// a Distance to 0'0".

Distance::Distance()
{
    feet = 0;
    inches = 0.0;
}

// Initialize a Distance to the specified value.

Distance::Distance( int ft, double in )
{
    feet = ft;
    inches = in;
}

// Define + that operates on Distance + Distance.

DistanceDistance::operator + ( Distance &rhs )
{
    int    sum_feet;
    double sum_inches;

```

```

    sum_feet = feet + rhs.feet;
    sum_inches = inches + rhs.inches;
    if (sum_inches >= 12.0)
    {
        sum_inches -= 12.0;
        ++sum_feet;
    }
    return Distance(sum_feet,sum_inches);
}

// Define + that operates on Distance + int,
// which means add feet.

DistanceDistance::operator + ( int rhs )
{
    return*this + Distance(rhs,0.0);
}

// Define + that operates on Distance + double,
// which means add inches.

DistanceDistance::operator + ( double rhs )
{
    return*this + Distance(0,rhs);
}

// Define + that operates on int + Distance,
// which means add feet.

Distanceoperator + ( int lhs, Distance &rhs )
{
    int    sum_feet;
    double sum_inches;

    sum_feet = lhs + rhs.feet;
    sum_inches = rhs.inches;
    return Distance(sum_feet,sum_inches);
}

// Define + that operates on double + Distance,
// which means add inches.

Distanceoperator + ( double lhs, Distance& rhs )
{
    returnrhs + lhs;
}

// Define stream insertion operator for Distances.

ostream& operator << ( ostream& str, Distance& d )
{
    str << d.feet << "" << d.inches << "\n";
    return str;
}

// Define stream extraction operator for Distances

istream& operator >> ( istream& str, Distance& d )
{
    str >> d.feet;
    if (str.get() != '\n')
    {
        cerr << "**** Error extracting Distance.\n";
        exit(1);
    }
    str >> d.inches;
    if (str.get() != '\n')

```

```
{  
    cerr << "*** Error extracting Distance.\n";  
    exit(1);  
}  
return str;  
}
```

Topic 3.8 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-3-dot-8>)